

Table of Contents

Table of Contents	1
Introduction	1
Purpose	1
Safety	1
Basic Arduino, Limit Switch, and Toggle switch	2
Equipment needed	2
Wiring Diagram	2
Arduino Connections	3
Connect the Arduino microcontroller board to the computer:	3
Verify that the computer can communicate with the Arduino	4
Exercise 1: Breadboard	4
Exercise 2: Limit Switch	5
Limit Switch Description	5
Connect the limit switch	6
Run the sketch for the limit switch	7
Exercise 3: Toggle Switch	8
Toggle Switch Description	8
Connect the toggle switch:	9
Run the sketch for the toggle switch	10
Exercise 4: Proximity Sensor	10
Proximity Sensor Description	10
Connect the proximity sensors	11
Run the sketch for the proximity sensors	12
Wiring Diagram	14
Clean Up	14
Lab 8 Post-Lab (as soon as your mechanism is ready)	14

Introduction

Purpose

Lab 1 Breakdown:

- Lab 1 Part 1: Basic Arduino, Limit Switch, and Toggle switch
- Lab 1 Part 2: Proximity Sensor
- Lab 1 Post-Lab: Install these components on your mechanism

In this lab you will learn how to use an Arduino microcontroller board, analog sensors, and digital sensors. You will wire the sensors to the microcontroller board and read the inputs from the sensors. This is the first of four labs dealing with sensors and microcontrollers. In the following labs you will add a motor and another digital sensor (Hall effect encoder) and use the information from all these sensors to control your system.

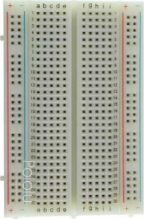
Safety

In this lab we will power all the sensors through the Arduino microcontroller board, which delivers very low current. Starting with the next lab session, we will use external power. This can damage parts and hurt you and bystanders. You should develop a habit to make sure that all connections are made properly, everyone is out of the way of any moving components, and the moving components are not in a position to run into any other objects before turning on power.

Basic Arduino, Limit Switch, and Toggle switch

Equipment needed

The following parts have been provided and are a part of your kit:

		
Limit Switch (a.k.a. Snap Action switch) (1)	Toggle Switch (1)	400-point Breadboard (1)

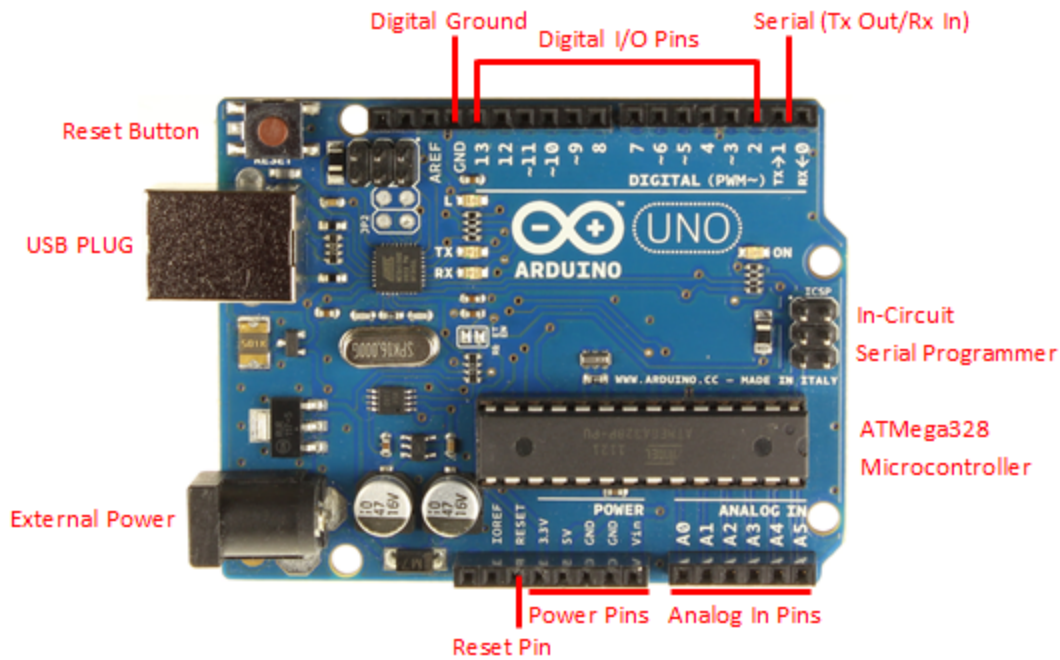


Figure 1: Arduino Uno Microcontroller Board

The following parts will be on the lab table in front of you. **You must keep them in the mechatronics lab since they will be used by the next ME350 section:**

- USB cable
- Extech multimeter
- wire strippers/pliers

The following parts will be on the lab table in front of you, and you can keep them with your kit.

- 10 kilo-ohm resistors (brown-black-orange-gold) (6)

The proximity sensors will be available in the Mechatronics Lab, **do not add them to your kit or remove them from the Lab** as all teams will share them for Lab 8.

Wiring Diagram

As you work through this lab, you should be making a wiring diagram (hand sketch) of all the connections you have made. Your diagram should show the Arduino, the breadboard, and all the sensors and switches. It should indicate which terminals on each device you have connected to which terminals on the other devices, and should also indicate the colors of the wires.

Arduino Connections

Connect the Arduino microcontroller board to the computer:

The Arduino software is already loaded onto the computers in the Mechatronics Lab.

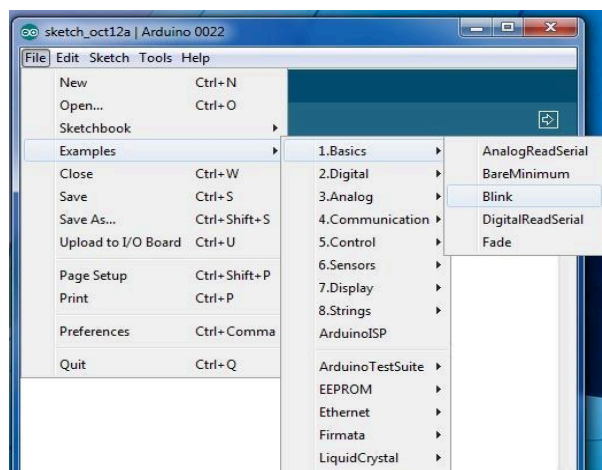
1. Log into the computer with your UMR00T username and password.

2. Plug the flat end of the USB cable into any USB port on the **Belkin USB hub**. Do **NOT** plug into the side of the monitor or the main computer tower. Plug the square end of the cable into the Arduino board.
3. Double-click the Arduino.exe icon on the desktop. This will bring up a sketch window that you can save later under any name you want. An Arduino program is called a sketch.
4. From the Tools menu, select Board, and verify that "Arduino Uno" is selected.
5. From the Tools menu, select Port, and verify that the serial port selected has (Arduino Uno) after COM#. For example, my selected port is "COM30 (Arduino Uno)"
6. From the Tools menu, select Serial Monitor, and change the baud rate (in the lower right corner) from the default 9600 baud to 115200 baud. The Serial Monitor can also be opened via the keyboard shortcut Ctrl + Shift + M or by clicking the  button in the top right corner of your sketch.
7. Look at the Arduino microcontroller board and identify the key regions as shown on the previous page.

Verify that the computer can communicate with the Arduino

To verify this and to demonstrate the procedure for uploading a sketch, we will use a very simple sketch from the Arduino library.

1. From the File menu, select Examples, Basics, Blink. The Blink sketch will appear in your window. Read the comments to get an idea of what the sketch does.



2. Press CTRL+U. This will compile the sketch and will upload it to the Arduino board. Notice on the Arduino board that the tiny LEDs (in the middle of the board) blink rapidly as the sketch is uploading. This is typical during uploading.
3. Once the sketch has been uploaded, it will run immediately. Look at the tiny LED near digital pin 13. It should blink on for one second, then off for one second, repeatedly. If it does not, double-check that you have completed all the steps above.
4. Congratulations! You just verified the computer can communicate with the Arduino.

Exercise 1: Breadboard

Wire needed (22 gauge, solid core):

one 1.5" red wire, one 2.5" red wire, one 1.5" black wire, one 2.5" black wire

1. Position the breadboard (as shown below), such that the letters are readable. Note the positive (red) bus and negative (blue) bus running vertically down the sides. We will use the buses on the left side in all of the lab experiments today.

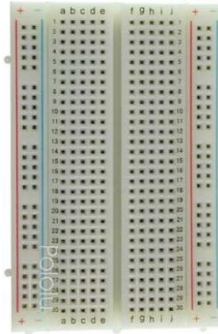


Figure 2: Correct Orientation of Breadboard

2. Cut a **red 1.5"** jumper wire (22 gauge, solid core) from the wire stack in the mechatronics lab. Strip about 1/3" of insulation from both ends of the wire. Connect one end to the positive (red) bus on the left hand side of your breadboard. Connect the other end to the 5V PWR pin on the Arduino.
3. Cut a **red 2.5"** jumper wire. Strip both ends about 1/3". Connect one end to the positive (red) bus on the left hand side of your breadboard (row 30) and connect the other end to the positive (red) bus on the right hand side of the breadboard. This transfers the power to the other bus.
4. Repeat steps 2 and 3 with the **black** jumper wires and the negative bus. Connect one end of the black 1.5" wire to the negative (blue) bus on the left hand side of the breadboard. Connect the other end to a GND pin on the Arduino (**any pin labeled GND is OK**). Use the **black 2.5" wire** to connect the negative (blue) busses on the left and right hand sides of the breadboard (row 30).
5. Plug in your Arduino and use your multimeter to verify that both busses have 5V. (One way to do this is to cut short test wires or pull them from the electronic waste bin. Plug one end of these wires into the bus and touch the other end to the probes of the multimeter to verify that the bus is at 5V. **Ensure the 5V and GND test wires do not touch** and don't forget to remove these test wires after you are finished to avoid shorting any component.)

We will be using this wiring convention:

Red = Positive Supply Voltage

Black wires = Ground

Yellow/White/Green/Blue Wires = Digital Signal

Keep the two power wires connected as you move on to the next steps.

Exercise 2: Limit Switch

Limit Switch Description



Figure 3: Limit Switch

The Omron Subminiature Basic Switch (Jameco part number 187733, shown in Figure 3) can detect the presence of an object within a very short range. The limit switch detects physical contact with the object. The object pushes the lever, which presses the button to close the switch. These types of switches are often used on industrial machinery to send feedback to the controller when a mechanism has reached the limit of its motion. **They do not act as a hard stop**, since the lever and the switch housing are not designed to absorb the load of the moving mechanism, but they can tell the controller when the mechanism has reached the hard stop. (In the next lab sessions, we will use them to *calibrate* your system with knowledge of the start or final position.)

Connect the limit switch

Wire needed (22 Gauge, Solid Core):

(1) 30" red wire, (1) 30" yellow wire, (1) 5.5" yellow wire, (1) 1" black wire

1. Notice that the three terminals on this limit switch are labeled C, NO, and NC. (If you don't see these markings, look at the other side of the switch.) These stand for common, normally open, and normally closed. For our application, we will connect one wire to the C terminal and one wire to the NO terminal to use the switch as normally open. That is, pressing the lever closes the circuit. Connecting to NC would have the opposite effect; the circuit would be interrupted when the lever is pressed.
2. To test the functionality of the switch, use your multimeter as a continuity tester. To do this, turn the dial to the Ω icon. Press the select button twice until the .))) icon shows in the display. Touch the red and black leads together to make sure that the multimeter beeps when it detects a closed circuit. Now check which of the three terminals on the switch are connected to each other:
 - when the lever on the switch is not pressed
 - when the lever on the switch is pressed
3. Using the 30" red wire, connect one end to the NO (center) terminal on the switch. This time, strip a little more than $\frac{1}{2}$ " of insulation from one end of the wire for the limit switch side and $\frac{1}{3}$ " of insulation from the other end for the breadboard. Use the pliers to bend the $\frac{1}{2}$ " side of wire around the terminal so it will not come loose. You should solder and heat shrink the connections to the limit switch by final testing.
4. Repeat with the 30" yellow wire and connect it to the C terminal on the switch.
5. Connect the other end of the 30" red wire to the positive bus on the breadboard. Make sure the Arduino's 5V pin is connected to the positive bus of the breadboard, from the previous experiment.
6. Connect the other end of the 30" yellow wire to row 1 (pin a) on the breadboard.

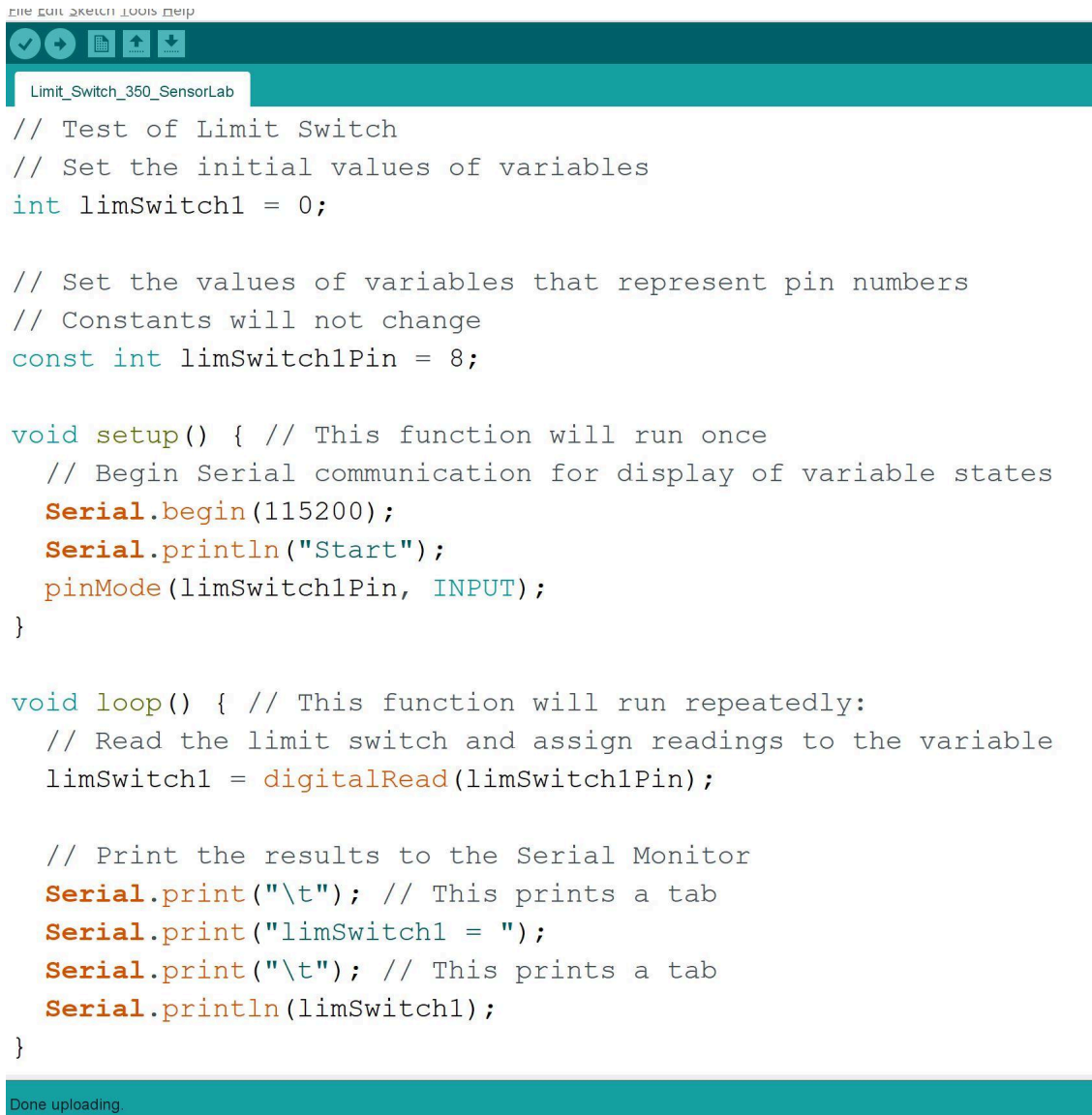
7. Using the **5.5" yellow wire**, strip both ends about 1/3" from the end. Connect one end to row 1 (pin b) on the breadboard and the other end to Arduino digital pin 8. This pin will be used to take the reading from the switch: HIGH (=5V) if the switch is closed and LOW (=0V) if the switch is open.
8. We need to add a pull-down resistor to our circuit. This resistor connects the pin to ground and ensures that its potential quickly goes to 0V when the switch is open. Take a 10 kilo-ohm resistor from your table. Bend the ends of both wires down at a 90 degree angle. Plug one end (it doesn't matter which one) into row 1 (pin e) and the other end into row 6 (pin e).
9. Using the **1" black wire**, strip both ends about 1/3" from the end. Connect row 6 (any pin from a to d) to the negative bus on the left hand side of the breadboard.
10. Congrats! You have successfully wired up one limit switch. If your team thinks it would be beneficial, you are allowed to use the same process to connect a second limit switch to your mechanism. This second limit switch could possibly be used to sense when your mechanism has contacted your second hard stop. If you choose to do this, you will need to wire the second limit switch on your own using the instructions above (use digital pin 11 on the Arduino) and will need to adapt the code accordingly to read two limit switches.

Run the sketch for the limit switch

1. Download [limSwitch_test.ino](#) script. Notice how the IDE (the compiler) turns certain words different colors to show that it recognizes them as standard commands. Pay attention to CAPITALIZATION because the commands and names of variables are case-sensitive.

If you have questions about the commands below, go to the www.Arduino.cc website and use the search engine there. The site also has a very good reference section describing all the commands.

Note: The sketch below uses the **Serial.println** command. This stands for "serial print line". The command prints data to the serial monitor window and then moves to the next line in the window. **The second to last character in this command is a lowercase L, not a capital i.**



```

// Test of Limit Switch
// Set the initial values of variables
int limSwitch1 = 0;

// Set the values of variables that represent pin numbers
// Constants will not change
const int limSwitch1Pin = 8;

void setup() { // This function will run once
  // Begin Serial communication for display of variable states
  Serial.begin(115200);
  Serial.println("Start");
  pinMode(limSwitch1Pin, INPUT);
}

void loop() { // This function will run repeatedly:
  // Read the limit switch and assign readings to the variable
  limSwitch1 = digitalRead(limSwitch1Pin);

  // Print the results to the Serial Monitor
  Serial.print("\t"); // This prints a tab
  Serial.print("limSwitch1 = ");
  Serial.print("\t"); // This prints a tab
  Serial.println(limSwitch1);
}

```

Done uploading.

2. Save your sketch (File, Save As) as Limit_Switch_350_SensorLab. Sketch names cannot contain spaces. **NOTE: Computers in the mechatronics lab may be reformatted without notice, so we recommend uploading all your sketches to Google Drive to avoid losing work.**
3. Use CTRL+U to upload the sketch to the Arduino. If you have any errors, it will let you know. If you get an error "Problem uploading to board", ask your GSI.
4. **NOTE: Open the serial monitor (see figure 4 on the next page) only after your sketch has finished uploading**, because when the computer communicates with the Arduino, it also uses serial communication. When you open the serial monitor the program uploaded to the Arduino will restart.
5. Open the serial monitor by clicking on the icon shown in Figure 4 or by using the shortcut keys CTRL+SHIFT+M. See the results as you press the switch. The serial monitor will continuously display the readings from the sensors, as a rolling printout in the window.
6. Close the serial monitor.
7. Congratulations, you just worked on your first Arduino program!

Keep your wiring as it is when you move on to the next exercise.

Note: If we had used the NC terminal instead of NO, our switch would be normally closed. We could still use the switch, but we would have to change our sketch to look for an open (LOW) condition to detect pressing of the lever. This configuration would be very useful for a fail-safe system where it is safety-critical to detect pressing of the lever (for example to stop machinery in an emergency). If the connection to the sensor would break (for example, due to a bad connection), the machine would automatically stop.

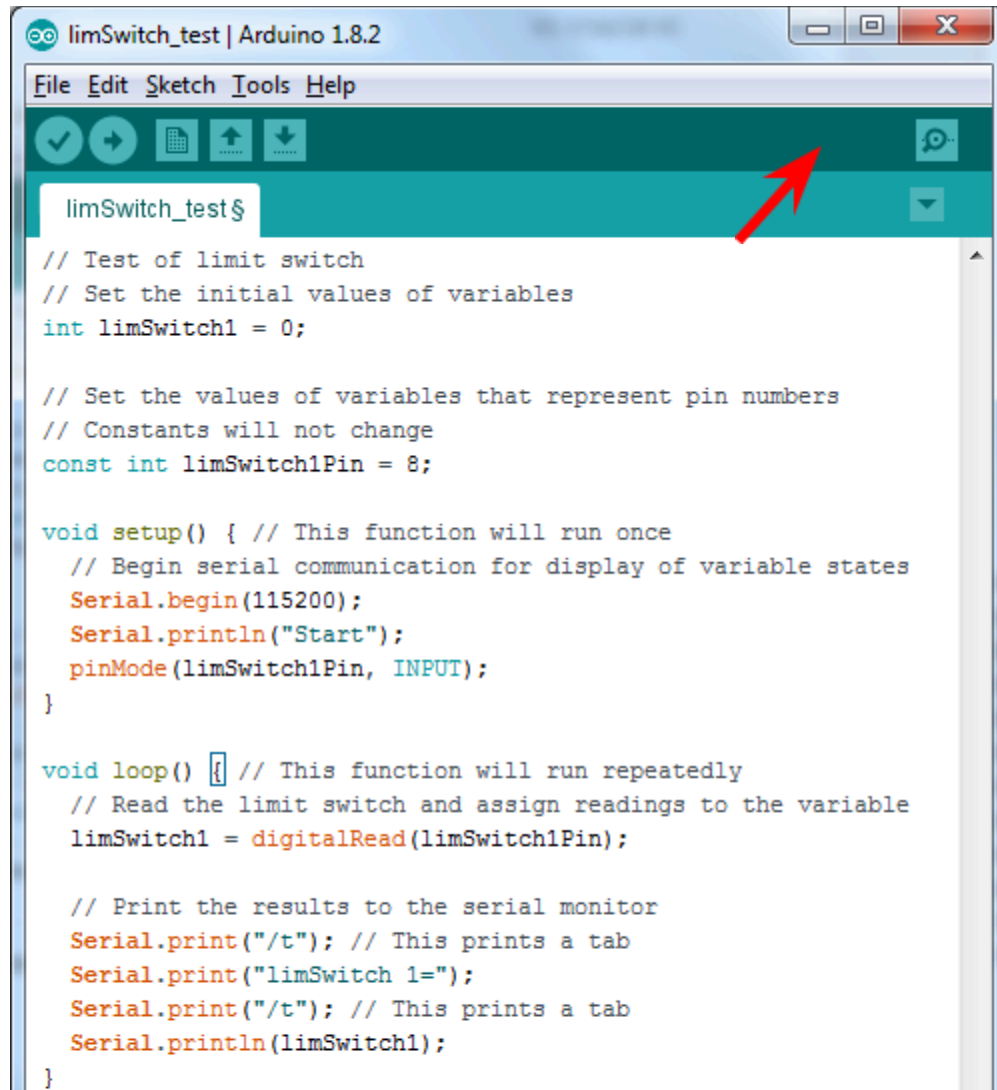


Figure 4: Icon to open the serial monitor window

Exercise 3: Toggle Switch

Toggle Switch Description



Figure 5: Toggle Switch

The toggle switch (Jameco part number 76523, shown in Figure 5) is a two-terminal ON/OFF switch. The circuit is connected when you press the toggle switch pole to the ON position and disconnected when you put the pole to the OFF position. (In your next lab session, you will use this switch to turn your linkage control system on and off.)

Connect the toggle switch:

Wire needed (22 Gauge, Solid Core):

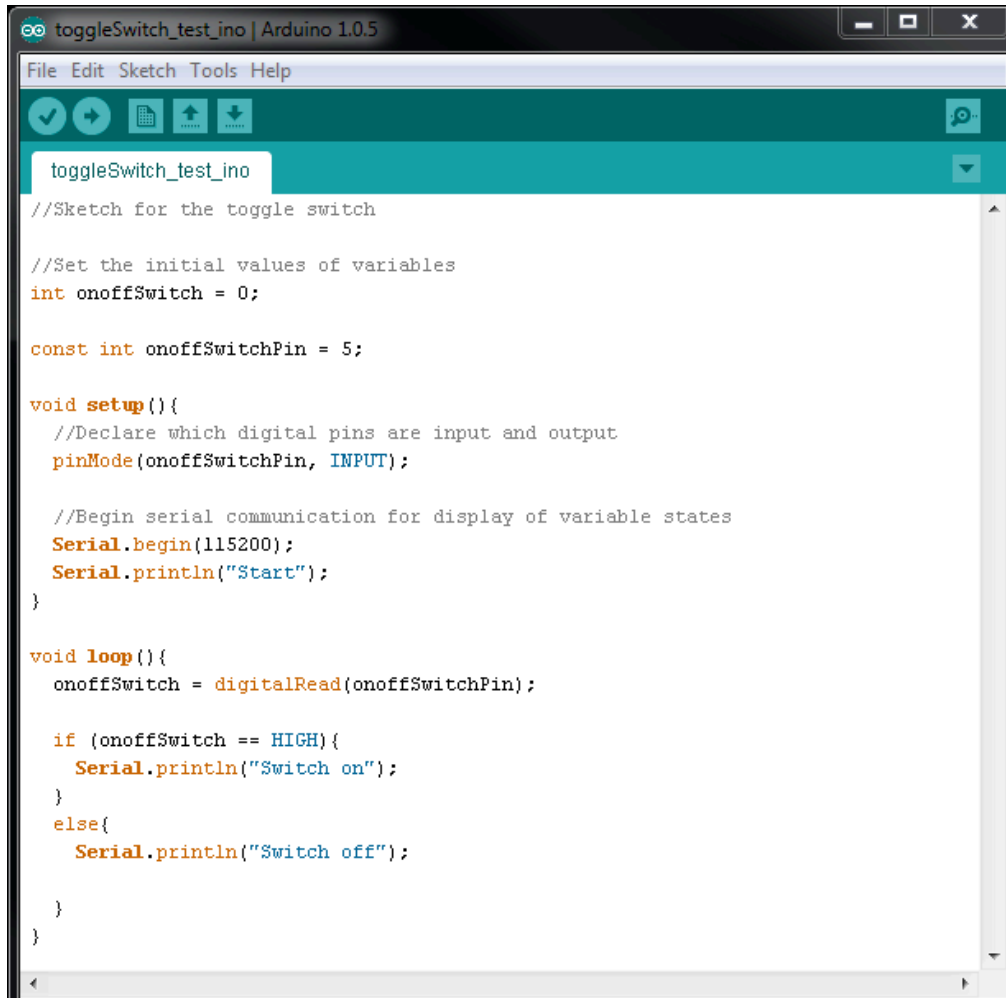
(1) 7" red wire, (1) 7" yellow wire, (1) 1" black wire, (1) 5.5" yellow wire

Unlike all the other sensors in this lab, the toggle switch receives input directly from the user, and needs to be located within the user's reach instead of located on the linkage system. Therefore, we will use shorter wires to connect to the toggle switch than to the limit switch.

1. The toggle switch has two terminals for attaching wires. Using your multimeter as a continuity tester, check whether the terminals are connected to each other when the pole flipped to each of the two positions. Determine which position of the pole is the ON position and which is OFF, relative to the terminals.
2. Using the 7" red wire, connect one terminal of the switch to the positive bus on the left hand side of the breadboard.
3. Using the 7" yellow wire, connect the other terminal of the toggle switch to row 8 on the left side of the breadboard. **(For this entire exercise, use any hole from a to e)**
4. As with the limit switch, we need a pull-down resistor. Connect a 10 kilo-ohm resistor between rows 8 and 12 on the breadboard.
5. To complete the circuit, use a 1" black wire to connect row 12 to the negative bus on the left hand side.
6. Connect the 5.5" yellow wire from row 8 on the breadboard to Arduino digital pin 5. This will receive the signal from the switch.

Run the sketch for the toggle switch

1. Download [toggleSwitch_test.ino](#).

A screenshot of the Arduino IDE interface. The title bar reads "toggleSwitch_test.ino | Arduino 1.0.5". The menu bar includes "File", "Edit", "Sketch", "Tools", and "Help". Below the menu bar is a toolbar with icons for checking, running, saving, and uploading. The main text area contains the following C++ code:

```
//Sketch for the toggle switch

//Set the initial values of variables
int onoffSwitch = 0;

const int onoffSwitchPin = 5;

void setup(){
  //Declare which digital pins are input and output
  pinMode(onoffSwitchPin, INPUT);

  //Begin serial communication for display of variable states
  Serial.begin(115200);
  Serial.println("Start");
}

void loop(){
  onoffSwitch = digitalRead(onoffSwitchPin);

  if (onoffSwitch == HIGH){
    Serial.println("Switch on");
  }
  else{
    Serial.println("Switch off");
  }
}
```

2. Save your sketch and upload it to the Arduino. **Remember to also save your sketch on Google Drive so it does not get erased if the computer gets reformatted.**
3. Open the serial monitor and see the results as you flip the switch.

Exercise 4: Proximity Sensor

Proximity Sensor Description

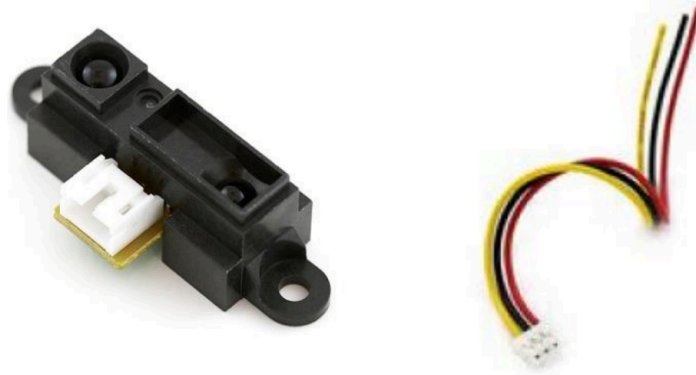


Figure 6: Infrared Proximity Sensor and 3-wire pigtail connector

The Sharp GP2D120XJ00F infrared proximity sensor (see Figure 6) is an analog sensor – it gives a variable output of voltage (from 3.1V to 0.3V depending on if the object is close or far away) instead of a simple high vs. low output. It detects the distance to a nearby object by shining a beam of infrared (IR) light from an LED, and measuring the intensity of reflected light using a phototransistor. In industry, these are often called “prox switches”. You can find the datasheet with detailed information on the sensor at this link:

<https://www.pololu.com/file/0J157/GP2D120-DATA-SHEET.pdf>.

Connect the proximity sensors

1. For this project, you will use four proximity sensors that are rigidly mounted to the playing field to detect the location of each target. Each proximity sensor can only detect one target, so we need four total proximity sensors (one for each rail). To wire the proximity sensors to your linkage, plug the provided 2x4 bus into your breadboard, making sure the side with the red wires is plugged into the positive breadboard bus and the side with the black wires is plugged into the negative bus.
2. Plug the yellow signal wire coming from the proximity sensor furthest from the mirror into analog pin A0 on the Arduino.
3. Plug the yellow signal wire coming from the proximity sensor second from the left into analog pin A1 on the Arduino.
4. Plug the yellow signal wire coming from the proximity sensor third from the left into analog pin A2 on the Arduino.
5. Plug the yellow signal wire coming from the proximity sensor closest to the mirror into analog pin A3 on the Arduino.

Run the sketch for the proximity sensors

1. Open the serial monitor and start a round of the playing field, following the instructions in the [ME 350 W25 Plant vs Zombies Board Use Guide](#). Watch the proximity sensor readings as the targets move back and forth. The closer a target is to the proximity sensor, the higher the value the sensor should read.
2. Download the [proxSensorTuning code](#). The purpose of this code is to determine the maximum and minimum proximity sensor readings for each rail. In this code, we read each proximity sensor 20 times in a row and store the values in four arrays of integers, named *proximitySenseXValues* where *X* is replaced with the proximity sensor number. We then average all the values in these arrays using the

function *average* (defined at the end of the code) and store the averages in the array *proximitySenseAvg*s. We average 20 values to reduce noise.

3. Your team is responsible for determining the maximum and minimum values at the top of the code. These variables are named according to the proximity sensor, and whether the value is a minimum or a maximum, so, for example, *PROXIMITYSENSE1_MAX* represents the maximum value that proximity sensor 1 would read (which would be when the target is furthest from the photosensor).

In order to find these proximity sensor values, your team needs to fill in arguments for the minimum and maximum functions in the code segment shown below. To update the minimum and maximum, you should compare the read in values with the previous minimum and maximum values.

```

115 // ***** UPDATE MIN AND MAX VALUES ***** //
116 // i represents the current proximity sensor, iterating through each one
117 for (int ProxSensor = 0; ProxSensor < sizeof(proximitySenseAvg)/sizeof(int); ProxSensor++) {
118     // TO DO: Fill in the arguments to the minimum function
119     // HINT: The values that were just read in are stored in the proximitySenseAvg array
120     PROX_MIN_MAX[ProxSensor][MIN] = min( , );
121
122     // TO DO: Fill in the arguments to the maximum function
123     // HINT: The values that were just read in are stored in the proximitySenseAvg array
124     PROX_MIN_MAX[ProxSensor][MAX] = max( , );
125 }

```

The current proximity sensor readings for each rail will be printed in the serial monitor. You can uncheck the “Autoscroll” option so you can more clearly see the values. Monitoring the serial output can help you ensure the proximity sensors are working correctly during your test.

4. Unplug and replug in the board to reset it to Round 1. After you have added the arguments for the minimum and maximum functions, upload the code to the Arduino. Shortly after the code has finished uploading, turn the motor power on and start the first round. Wait for the round to finish running. A couple seconds after the round finishes, the Arduino will print out the minimum and maximum values for each proximity sensor. Now, use the printed values to update the proximity sensor limits shown below. **Note: You have to enter the limits as integers, not floats.**

```

3 // Definition of proximity sensor limits for each target:
4 const int PROXIMITYSENSE1_MAX = 0; // [proximity sensor counts] Value of prox sensor 1 when zombie is closest to photosensor
5 const int PROXIMITYSENSE1_MIN = 0; // [proximity sensor counts] Value of prox sensor 1 when zombie is closest to prox sensor
6 const int PROXIMITYSENSE2_MAX = 0; // [proximity sensor counts] Value of prox sensor 2 when zombie is closest to photosensor
7 const int PROXIMITYSENSE2_MIN = 0; // [proximity sensor counts] Value of prox sensor 2 when zombie is closest to prox sensor
8 const int PROXIMITYSENSE3_MAX = 0; // [proximity sensor counts] Value of prox sensor 3 when zombie is closest to photosensor
9 const int PROXIMITYSENSE3_MIN = 0; // [proximity sensor counts] Value of prox sensor 3 when zombie is closest to prox sensor
10 const int PROXIMITYSENSE4_MAX = 0; // [proximity sensor counts] Value of prox sensor 4 when zombie is closest to photosensor
11 const int PROXIMITYSENSE4_MIN = 0; // [proximity sensor counts] Value of prox sensor 4 when zombie is closest to prox sensor

```

5. Once you have finished tuning the thresholds, be sure to **save your sketch on Google Drive so it does not get erased if the computer gets reformatted.**

Wiring Diagram

1. Verify the wiring diagram (hand sketch) you have made according to the instructions. **Keep this diagram instead of handing it in. You will refer to it in your next lab session, and you will also use it for your final project report. (You can rewrite it for the final report.)**

Clean Up

1. Save your four sketches to a flash drive or upload them to Google Drive. You will use them at the beginning of your next lab session to check your wiring connections.
2. Use the dustpan and brush (see the GSI if you cannot find them) to clean up any stripped ends of wire insulation from your work area.
3. Keep the wires in the Arduino microcontroller board. Keep the wires and resistors in the sensors and breadboard. You will use them for your next lab session. Keep the Arduino attached to the Plexiglas mounting board. Take your parts with you.
4. **Leave the following items on the lab table, the same way that you found them, so they can be used by the next ME350 section:**
 - o USB cable
 - o Extech multimeter
 - o wire strippers/pliers

Lab 8 Post-Lab (as soon as your mechanism is ready)

- 1) **Install bracket to mount the limit switch onto your ground link.** The **limit switch** should be mounted in a location where the lever will be moved by your mechanism (preferably the input or output link) just as it contacts the hard stop in target position 1. We will use it to *calibrate* your encoder with knowledge of the start position. The limit switch is not meant to function as a mechanical positive stop. To mount the switch, use screws to attach it to tapped holes or double-stick tape to attach it to existing components. **Do not use hot glue as it tends to become loose and cause calibration issues. Screws are highly recommended.** (To drill the holes, use the drill press if possible instead of the milling machines.)